

Deploying a complete Machine Learning fraud detection solution using Amazon SageMaker

<https://catalog.workshops.aws/sagemaker-fraud-detection/en-US>

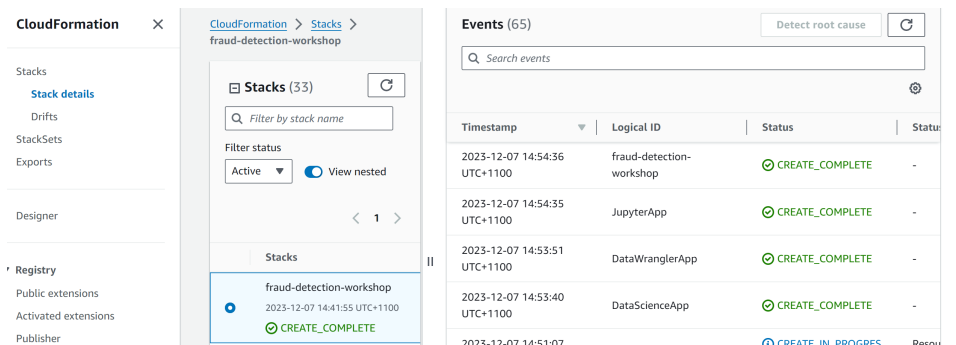
Overview

According to the [Insurance Information Institute \(III\)](#) , insurance fraud is a deception committed against an insurance company for financial gain. It can be anything from lying about a garaging address (the location where your vehicle is parked for most of the year) to exaggerating or outright staging an accident. Auto-insurance fraud is illegal in all 50 states in the United States of America, and insurance companies work hard to investigate and expose fraudulent insurance claims.

The purpose of this workshop is to demonstrate how to **build, train, and deploy** an end-to-end Machine Learning solution that helps us detect fraudulent auto insurance claims.

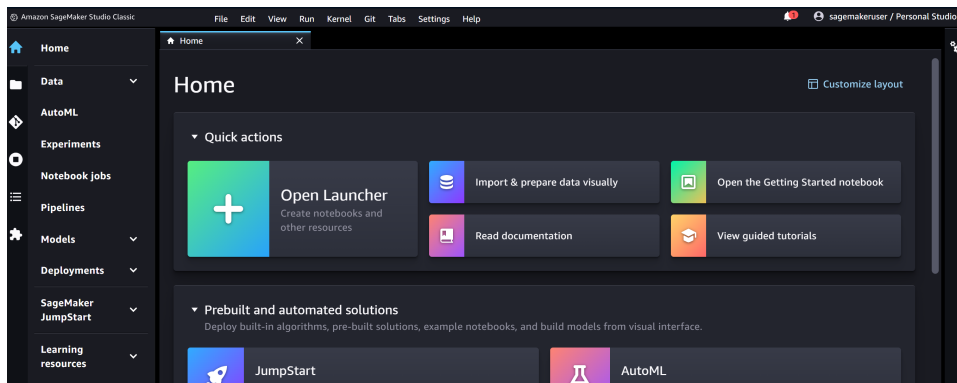
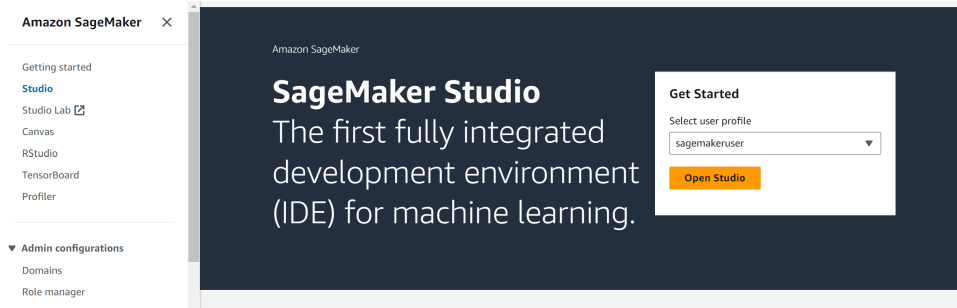
The workshop can be executed by anyone who would like to know more about Machine Learning on AWS. You need basic Python skills to navigate the workshop.

Data used in this workshop is synthetically generated and consists of customer biographical data, and auto insurance claims data. On the onset, it is not ready for Machine Learning and requires transformation before it is ready for training. We will also create a Machine Learning Pipeline to automate the training, tuning and deployment stages for an automated end-to-end solution.



The screenshot displays the AWS CloudFormation console. On the left, the 'CloudFormation' sidebar shows the 'Stacks' section. The main panel shows the 'fraud-detection-workshop' stack, which is in the 'CREATE_COMPLETE' state. Below the stack details, a table lists the events for this stack, showing the successful creation of the 'fraud-detection-workshop', 'JupyterApp', 'DataWranglerApp', and 'DataScienceApp' resources.

Timestamp	Logical ID	Status	Reason
2023-12-07 14:54:36 UTC+1100	fraud-detection-workshop	CREATE_COMPLETE	-
2023-12-07 14:54:35 UTC+1100	JupyterApp	CREATE_COMPLETE	-
2023-12-07 14:53:51 UTC+1100	DataWranglerApp	CREATE_COMPLETE	-
2023-12-07 14:53:40 UTC+1100	DataScienceApp	CREATE_COMPLETE	-
2023-12-07 14:51:07		CREATE_IN_PROGRESS	Resource



Explanation of the code files

There are five notebooks in the folder `FraudDetectionWorkshop`:

File Name	Description
Lab_1_and_2-Data-Exploration-and-Features.ipynb	Notebook with Lab 1 & 2
Lab_3_and_4-Training_and_Deployment.ipynb	Notebook with Lab 3 & 4
Lab_5-Pipelines.ipynb	Notebook with Lab 5 (SageMaker pipelines)
Lab_Bonus-Batch_Transform.ipynb	Bonus notebook demonstrates Batch Transform
Lab_Bonus-Detect_bias_using_SageMaker_Clarify.ipynb	Bonus notebook demonstrates Bias detection

Explanation of the data files

The target data exists in the `data` directory. Below is a list of files and their brief description.

File Name	Description
claims_customers.csv	Horizontally combined dataset of customer and claims dataset
claims_preprocessed.csv	Contains claims data after Data Preparation step
claims.csv	Contains all claims fraudulent and non-fraudulent
customers_preprocessed.csv	Contains customer data after Data Preparation step
customers.csv	Contains information about the customers
dataset.csv	Contains merged dataset (claims and customers) after Data Preparation step (features ready to the training)
test.csv	Contains the data we'll use to test the model
train.csv	Contains the data we'll use to train the model
upsampled_data.csv	This is the dataset pre-created after fixing the bias. Used with SageMaker Clarify

Explanation of output files

The `outputs` directory contains two files that contain data transformations. We will use these files in Lab 5.

- `claims_flow_template`
- `customer_flow_template`

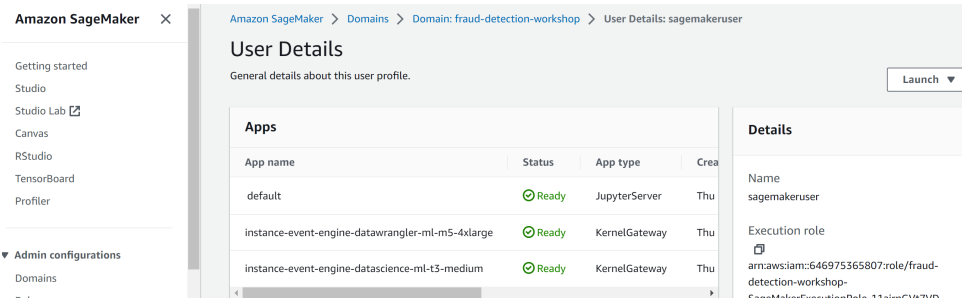
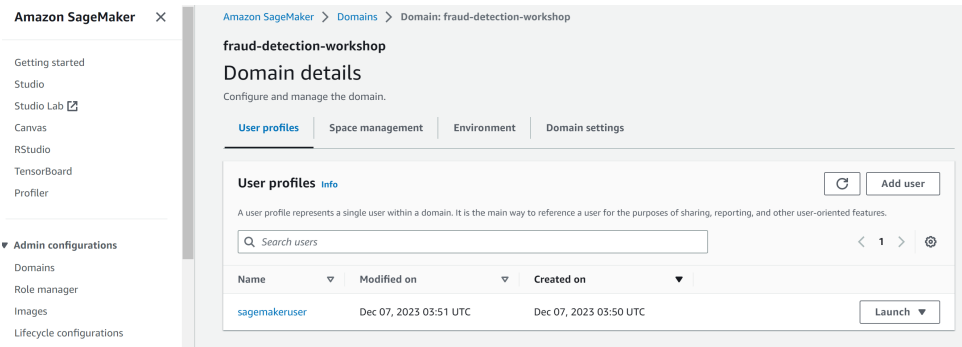
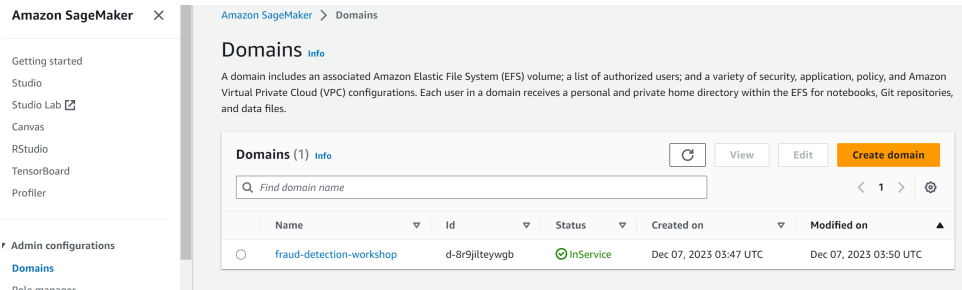
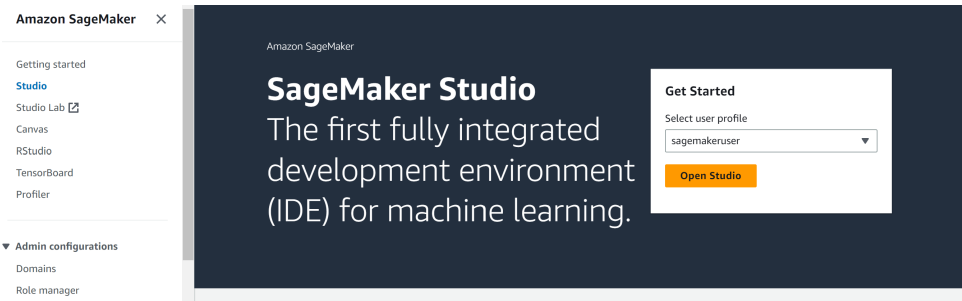
Explanation of helper scripts

There are seven helper scripts in `scripts` directory:

File Name	Description
<code>create_dataset.py</code>	Create train and test datasets
<code>create_feature_store.py</code>	Create Feature Stores, loads sampled data and waits for data to be available offline
<code>demo_helpers.py</code>	Useful functions to delete project resources and general functions
<code>evaluate.py</code>	Evaluation script for measuring model accuracy
<code>inference_specification.py</code>	Defines how to perform inference generation after a training job is run
<code>lambda_deployer.py</code>	Lambda function that will be created on pipeline to execute model deployment
<code>xgboost_starter_script.py</code>	A Python script that is executed as the training entry point

XX

All Pics



Lab_1_and_2-Data-Exploration-and-Features.ipynb

Lab1 and Lab2: Data Transformation and Feature Engineering

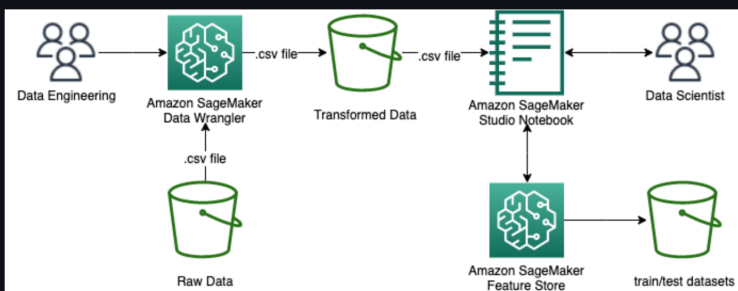
Introduction

There are two datasets that make up the inputs to our models. The first is insurance claim data and the other is customer data. This data is *synthetically generated*.

Raw dataset visualization and manipulation will be done using the following libraries: `seaborn` and `pandas`.

We will then generate features and store them using Amazon SageMaker Feature Store.

This diagram summarizes the next steps



```
df_claims.head()
```

```
[4]:
```

	incident_severity	num_vehicles_involved	num_injuries	num_witnesses	police_report_available	injury_claim	vehicle_claim	total_claim_amount	incident_id
policy_id									
1	1.0	3	0	2	1.0	0	15000	15000	
2	1.0	2	3	0	0.0	30000	19500	49500	
3	0.0	2	0	1	0.0	0	10000	10000	
4	0.0	1	0	0	0.0	0	14000	14000	
5	0.0	2	0	2	0.0	0	12000	12000	

5 rows x 29 columns

```
[5]: # cell 5
```

```
# and the customer dataset
df_customers.head()
```

```
[5]:
```

	customer_age	customer_education	months_as_customer	policy_deductable	policy_annual_premium	policy_liability	auto_year	num_claims_past_year	n
policy_id									
1	54	2	381	750	3000	1	2019	0	
2	41	3	31	750	3000	3	2017	0	
3	57	3	84	750	3000	3	2014	0	
4	74	4	123	750	2600	2	2018	0	
5	39	1	34	750	3000	2	2019	0	

```
df_combined.head()
```

[7]:

	policy_id	incident_type_theft	policy_state_ca	policy_deductable	num_witnesses	policy_state_or	incident_month	customer_gender_female	num_insurers_past
0	1675	0	0	750	0	0	2	0	
1	9	0	0	750	0	0	9	0	
2	1687	0	1	750	0	0	7	1	
3	1687	0	1	750	0	0	7	0	
4	1692	0	0	750	2	0	6	1	

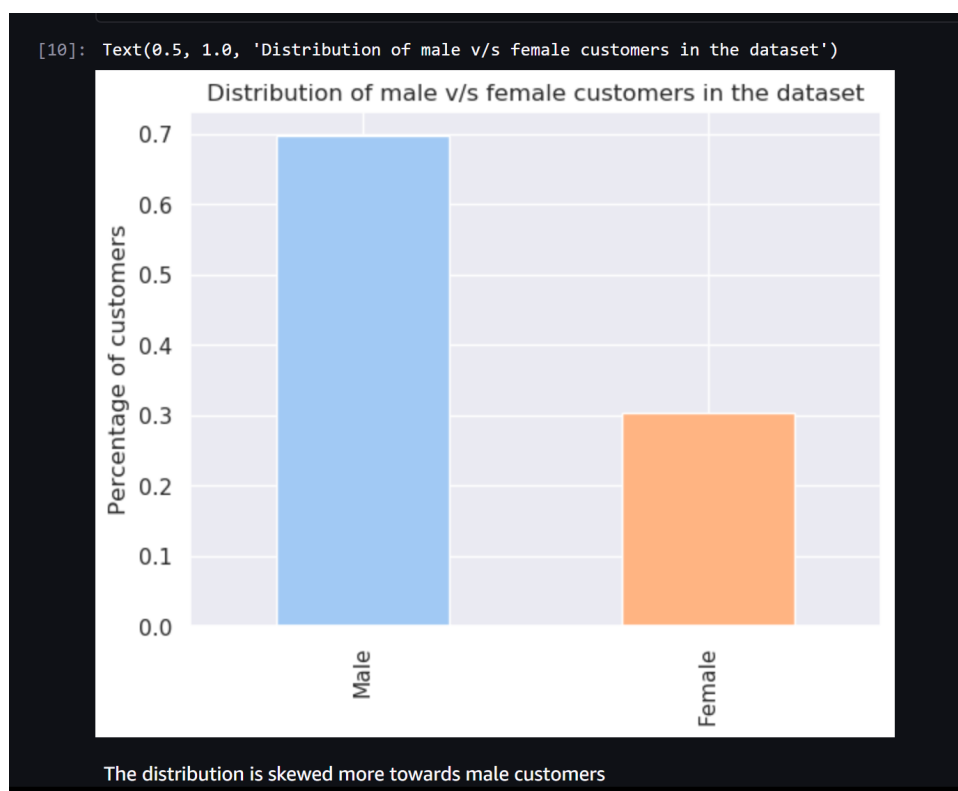
5 rows × 47 columns

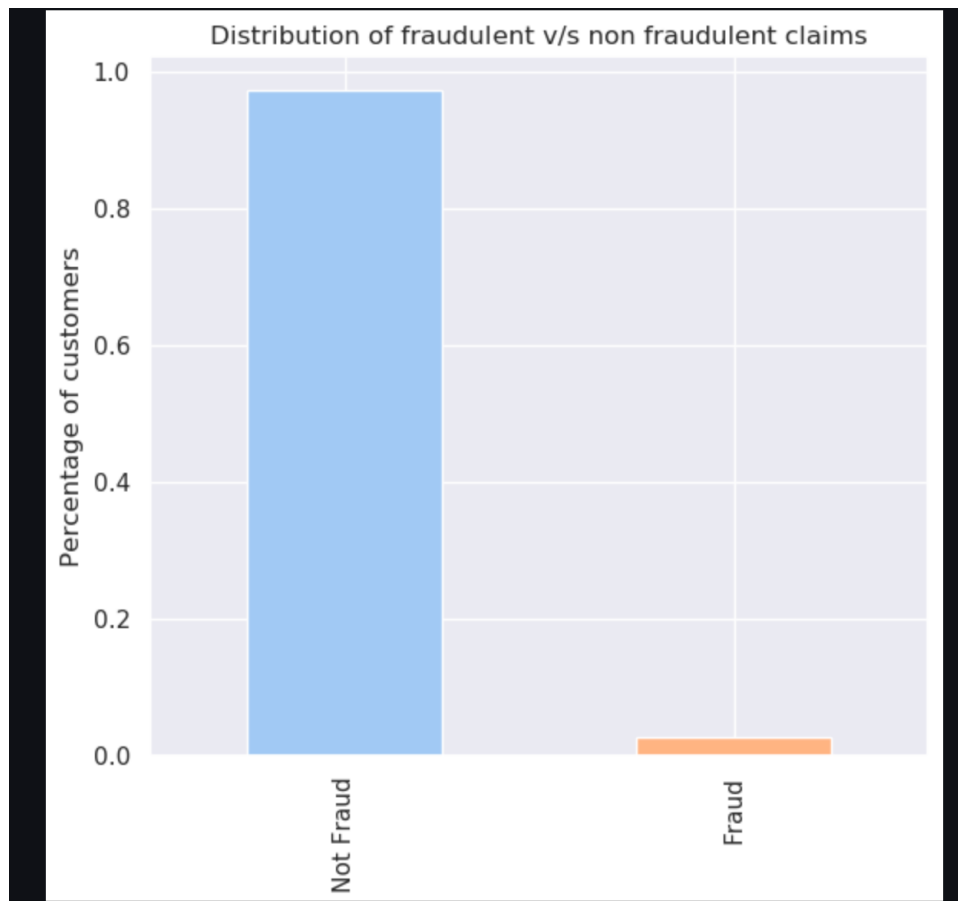
```
df_combined.describe()
```

[8]:

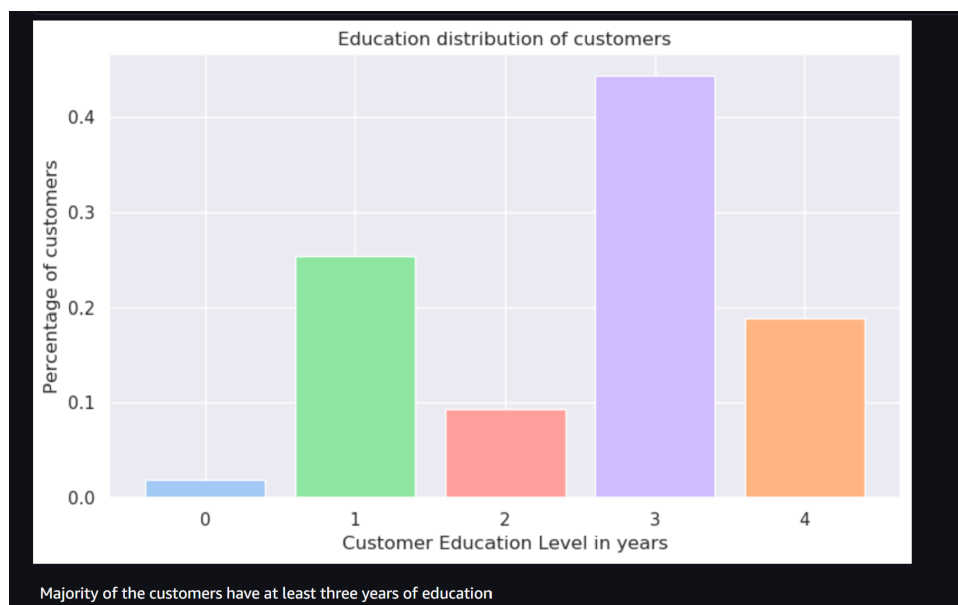
	policy_id	incident_type_theft	policy_state_ca	policy_deductable	num_witnesses	policy_state_or	incident_month	customer_gender_female	num_insure
count	20000.00000	20000.000000	20000.0000	20000.00000	20000.000000	20000.000000	20000.000000	20000.000000	
mean	2500.50000	0.048200	0.6204	751.13000	0.866100	0.070000	6.713200	0.372400	
std	1443.41173	0.214194	0.4853	13.57322	1.097921	0.255153	3.654396	0.483456	
min	1.00000	0.000000	0.0000	750.00000	0.000000	0.000000	1.000000	0.000000	
25%	1250.75000	0.000000	0.0000	750.00000	0.000000	0.000000	3.000000	0.000000	
50%	2500.50000	0.000000	1.0000	750.00000	0.000000	0.000000	7.000000	0.000000	
75%	3750.25000	0.000000	1.0000	750.00000	2.000000	0.000000	10.000000	1.000000	
max	5000.00000	1.000000	1.0000	1100.00000	5.000000	1.000000	12.000000	1.000000	

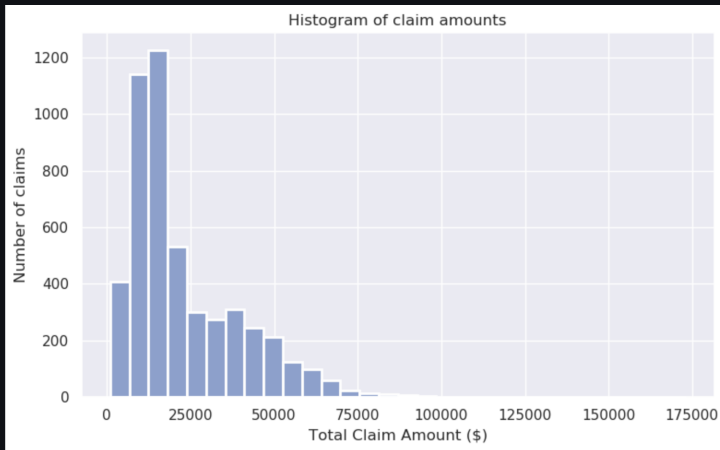
8 rows × 47 columns



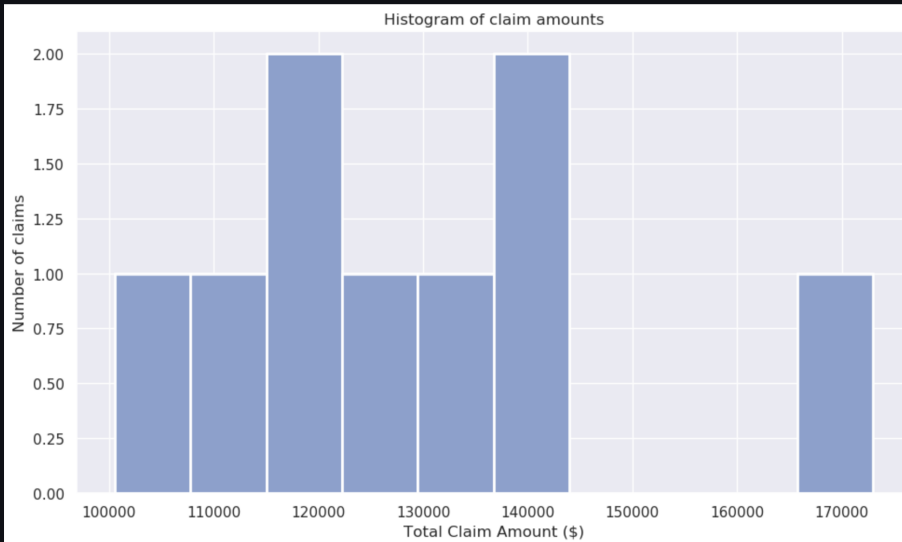


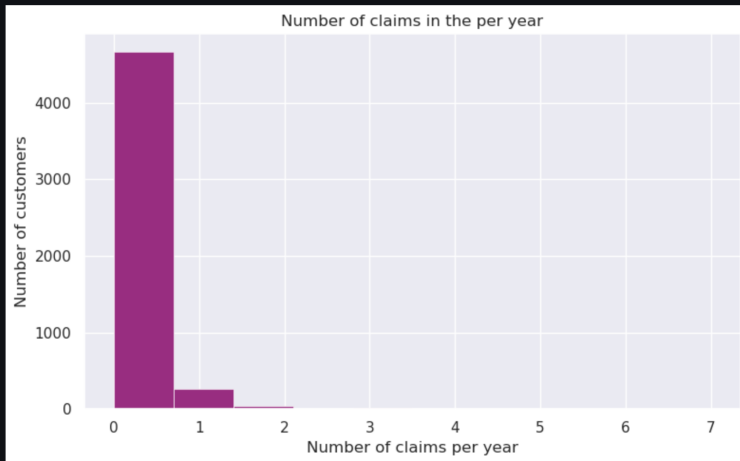
Majority of the claims aren't fraudulent which is a good sign



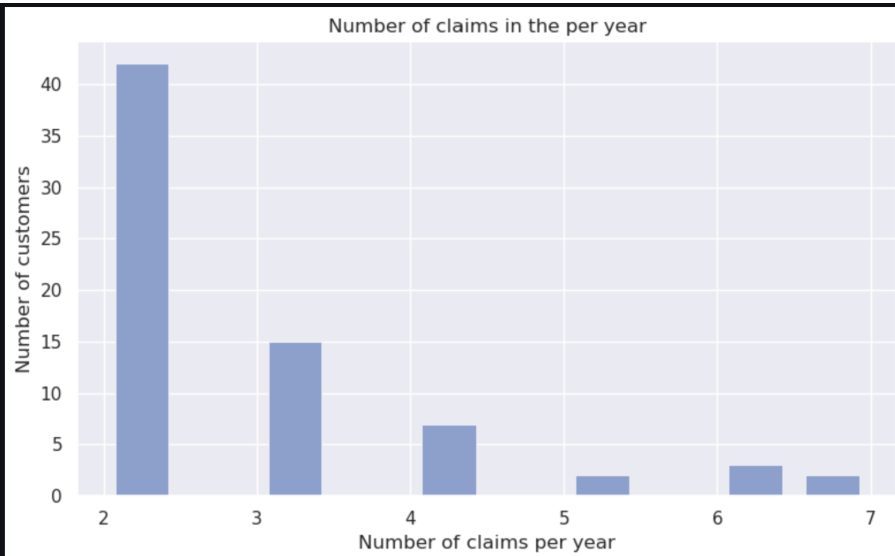


Majority of the total claim amounts are under "\$25,000". The higher claim amounts on the x-axis show up in this chart but they have very few numbers of claims against them. Hence, nothing is visible against them on the y-axis.

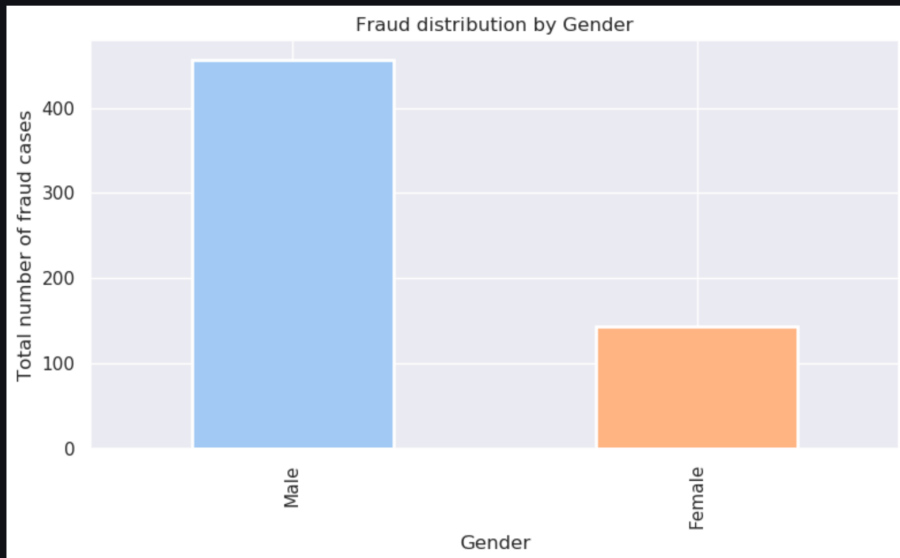




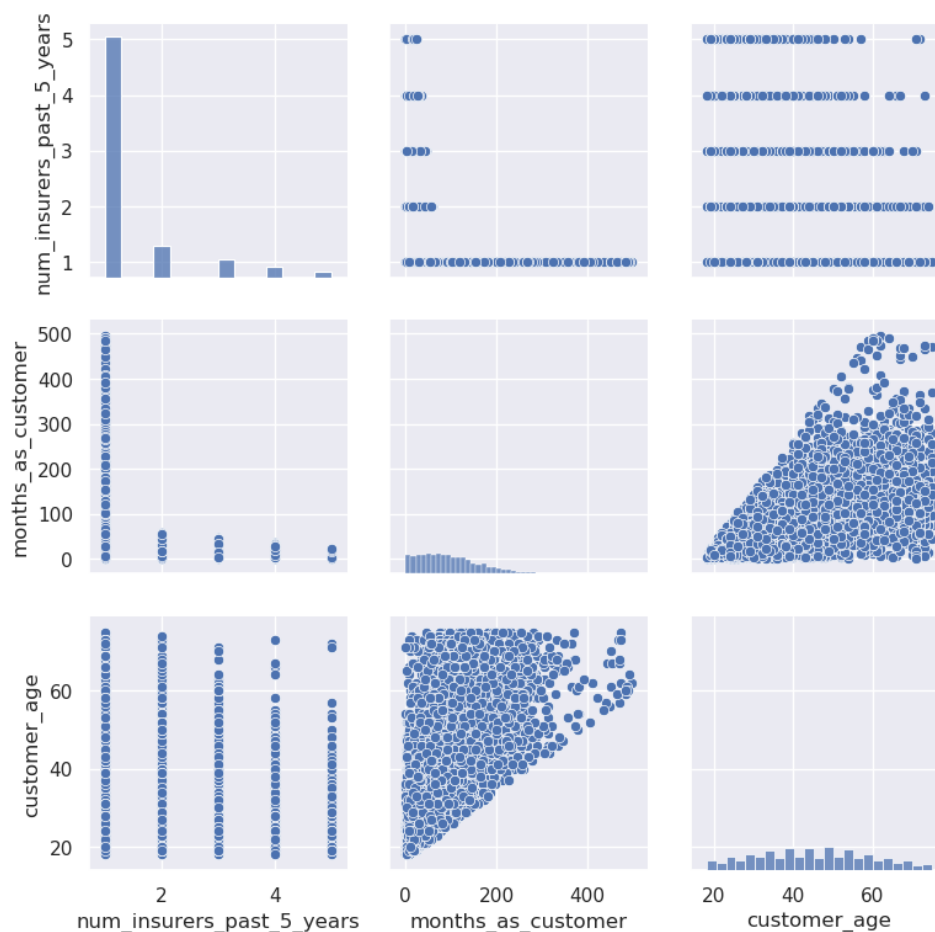
Most customers did not file any claims in the previous year, but some filed as many as 7 claims. Even though the chart doesn't show a Y value against claims 3 - 7, the fact that these value show up on the x-axis means there are some customers who did file 2 to 7 claims.

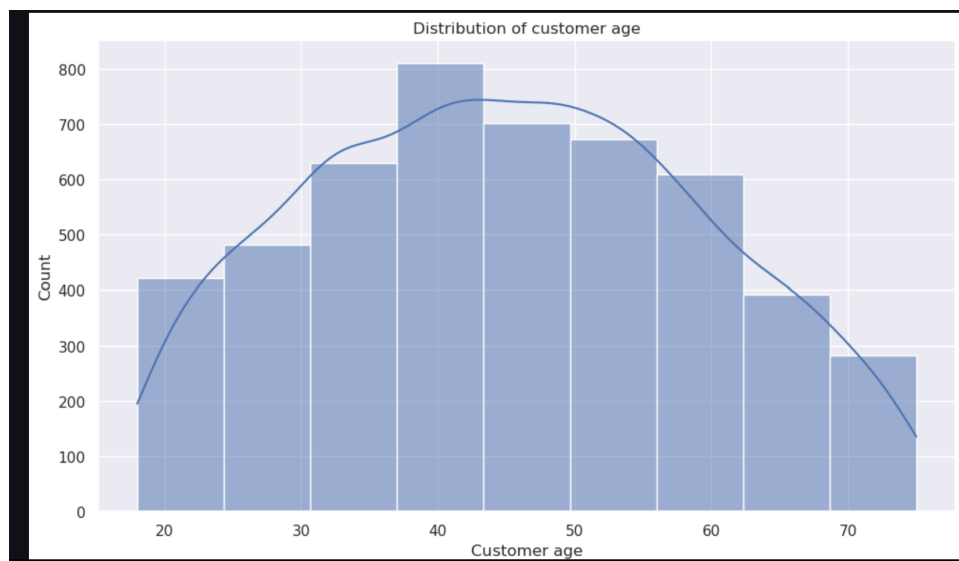
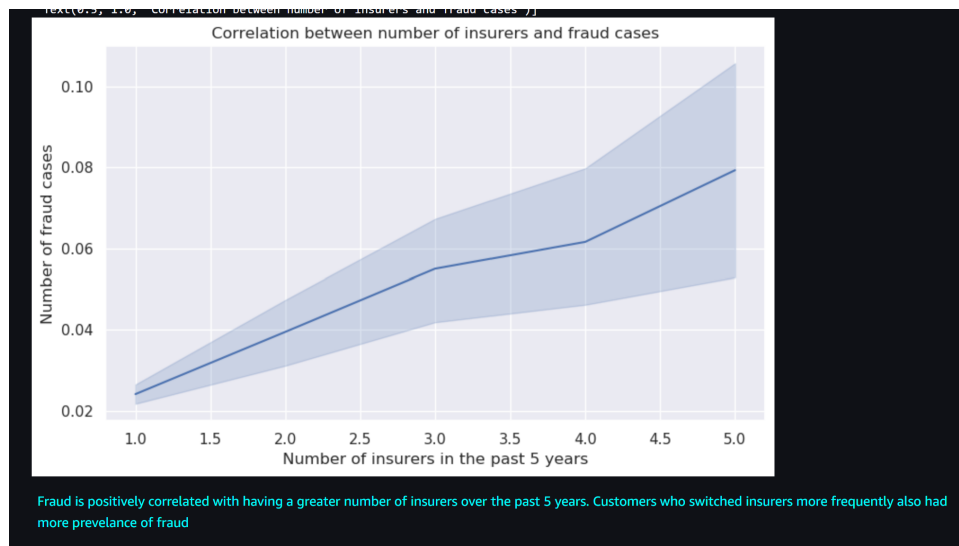


As mentioned, a very few number of people filed 2 to 7 claims compared to 4500+ who never a filed a claim



Fraudulent claims seem to come disproportionately from male customers.



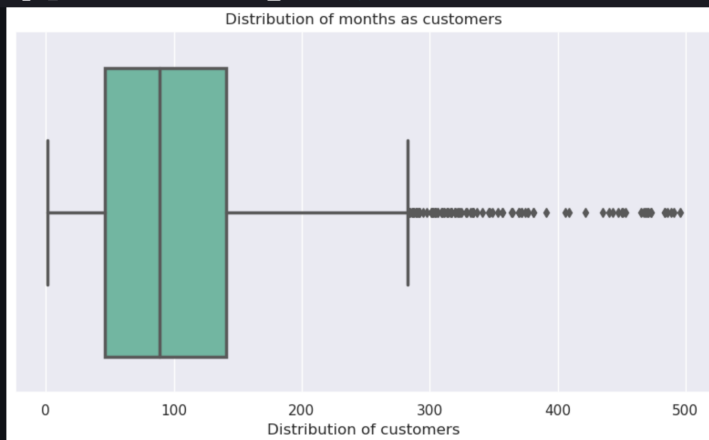


The age distribution is somewhat symmetric. We see a higher density of customers between the ages of 35 to 50. This can be further confirmed by the `describe` call on the `df_customers` dataframe `customer\_age` column.

```
df_customers.customer_age.describe()
```

```
[22]: count    5000.000000  
      mean      45.085400  
      std       14.290254  
      min       18.000000  
      25%       34.000000  
      50%       45.000000  
      75%       56.000000  
      max       75.000000  
      Name: customer_age, dtype: float64
```

75% of the users are less than or equal to 56 years old

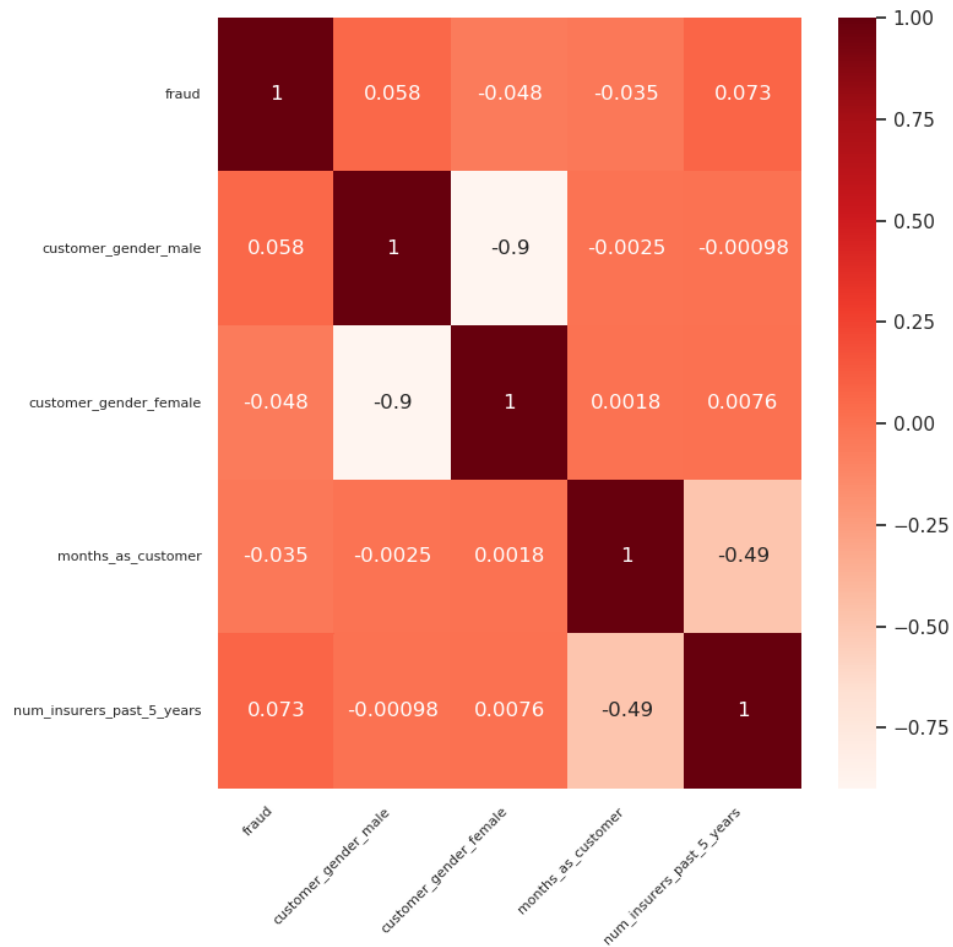


In this box plot we can see there are datapoints outside of the distribution. **These are outliers**. The median in this plot shows that the median "months as customer" is less than 100 (~83y).

The majority of the customers have been customers from 50 to 150 months. Again, this can be confirmed if we use the `describe` call on the `df_customers` dataframe

```
df_customers.months_as_customer.describe()
```

```
[24]: count    5000.000000  
      mean     101.328200  
      std      73.718143  
      min       1.000000  
      25%      46.000000  
      50%      89.000000  
      75%     141.000000  
      max     496.000000  
      Name: months_as_customer, dtype: float64
```



Fraud is correlated with having more insurers in the past 5 years, and negatively correlated with being a customer for a longer period of time. These go hand in hand and means that long time customers are less likely to commit fraud.

Other statistical insights:

- Negative correlation between `fraud` and female customers and positive correlation between `fraud` and the male gender. This insight was confirmed in our previous section with barcharts.
- Looking at these correlations some may look obvious but correlation charts give a good idea of magnitude of the relationship. What that means is it's one thing to instinctively know that if a certain metric increases the other positively correlated metric will increase as well. However, the rate at which it grows also plays an important part in the analysis.



- Relatively higher positive correlation between `total_claim_amount` and `injury_claim` than `total_claim_amount` and `vehicle_claim`.
- `incident_severity` relatively higher in positive correlation with `vehicle_claim` than `injury_claim` - this indicates that the severe incidents very likely saw higher cases of vehicle claims and fewer of injury claims.

Feature Store

Show introduction

Feature Group Catalog

Feature Catalog

Search by Feature group name or Description or Re

Source

My account

Cross account

Create feature group

Feature group name	Description	Tags	Record identifier	Store configuration	Status	Created on ↓
fraud-detect-dem...		2 items	policy_id	Online/Offline	Created	12/07/2023
fraud-detect-dem...		2 items	policy_id	Online/Offline	Created	12/07/2023

End of results

2 results

Refresh

Rows10

Go to page1

Page 1

Feature Store

Show introduction

Feature Group Catalog

Feature Catalog

Search by Feature name or Feature group name or I

Source

My account

Cross account

Feature name	Feature group name	Type	Description	Parameters	Created on ↓
customer_age	fraud-detect-demo-cu...	Integral			12/07/2023
policy_state_wa	fraud-detect-demo-cu...	Integral			12/07/2023
auto_year	fraud-detect-demo-cu...	Integral			12/07/2023
num_claims_past_year	fraud-detect-demo-cu...	Integral			12/07/2023
customer_gender_male	fraud-detect-demo-cu...	Integral			12/07/2023
customer_gender_fem...	fraud-detect-demo-cu...	Integral			12/07/2023
months_as_customer	fraud-detect-demo-cu...	Integral			12/07/2023

test.head(5)

[47]:

	fraud	num_insurers_past_5_years	policy_state_ca	incident_day	customer_age	incident_type_collision	driver_relationship_child	num_witnesses	policy_state_id
0	0		1	0	9	57	1	1	1
7	0		1	1	24	45	1	0	2
21	0		1	0	5	31	1	0	1
24	0		3	1	14	23	1	0	0
25	0		1	1	11	48	1	0	0

5 rows × 46 columns

Lab_3_and_4-Training_and_Deployment.ipynb

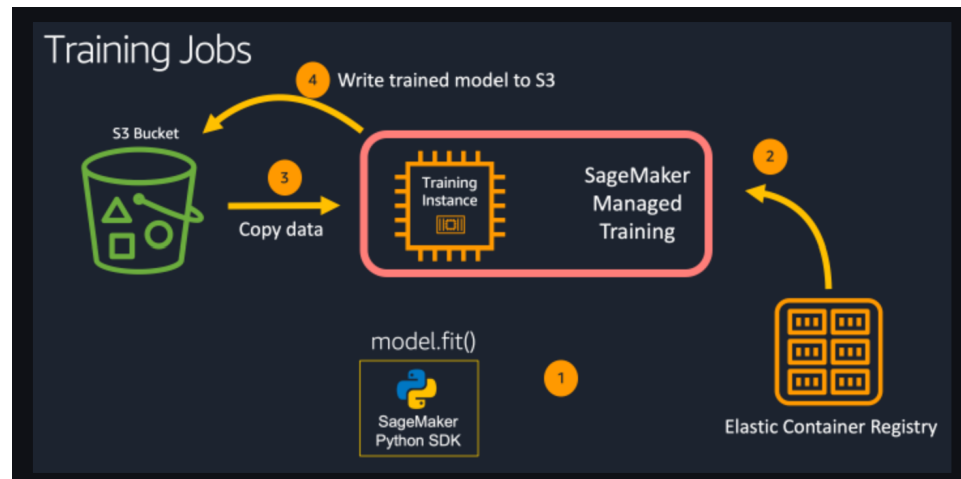
Lab 3: Train a Model using XGBoost

To train a model in SageMaker, you create a training job. The training job includes the following information:

1. The URL of the Amazon Simple Storage Service (Amazon S3) bucket where you've stored the training data.
2. The compute resources that you want SageMaker to use for model training. Compute resources are ML compute instances that are managed by SageMaker.
3. The URL of the S3 bucket where you want to store the output of the job.
4. The Amazon Elastic Container Registry path where the training code is stored.

Lets walkthrough how the training process works:

1. Invoke `model.fit()` to start the training process
2. Process downloads your training algorithm from ECR
3. Training data from S3 is used to train the model and create the model file
4. Model file is written to S3 which will be used as input for inferencing in the Deploy process



Once the training and test datasets have been persisted in S3, you can start training a model by defining which SageMaker Estimator you'd like to use. For this guide, you will use the XGBoost Open Source Framework to train your model. This estimator is accessed via the SageMaker SDK, but mirrors the open source version of the XGBoost Python package. Any functionality provided by the XGBoost Python package can be implemented in your training script. This estimator includes reference to the relevant training algorithm from ECR.

XGBoost is an extremely popular, open-source package for gradient boosted trees. It is computationally powerful, fully featured, and has been successfully used in many machine learning competitions.

Set HyperParameters ¶

These are the parameters which will be sent to our training script in order to train the model. Although they are all defined as "hyperparameters" here, they can encompass XGBoost's Learning Task Parameters, Tree Booster Parameters, or any other parameters you'd like to configure for XGBoost.

For this example, we will use the following hyperparameters for the XGBoost algorithm:

- `max_depth` - Controls how deep each tree within the algorithm can be built. Deeper trees can lead to better fit, but are more computationally expensive and can lead to overfitting. There is typically some trade-off in model performance that needs to be explored between a large number of shallow trees and a smaller number of deeper trees.
- `num_round` - Controls the number of boosting rounds. This is essentially the subsequent models that are trained using the residuals of previous iterations. More rounds should produce a better fit on the training data, but can be computationally expensive or lead to overfitting.
- `eta` - Controls how aggressive each round of boosting is. Larger values lead to more conservative boosting.
- `objective` - Specifies the learning task and the corresponding learning objective. Use `binary:logistic` for binary classification task.

▼ Training: Create and fit the estimator

If you want to explore the breadth of functionality offered by the SageMaker XGBoost Framework you can read about all the configuration parameters by referencing the inheriting classes. The XGBoost class inherits from the Framework class and Framework inherits from the EstimatorBase class:

- XGBoost Estimator documentation
- Framework documentation
- EstimatorBase documentation

For this example, we will use the following parameters for the XGBoost estimator:

- `entry_point` - Path to the Python source file which should be executed as the entry point to training.
- `hyperparameters` - Hyperparameters that will be used for training. The hyperparameters are made accessible as a `dict[str, str]` to the training code on SageMaker.
- `output_path` - S3 location for saving the training result (model artifacts and output files).
- `framework_version` - XGBoost version you want to use for executing your model training code.
- `instance_type` - Type of EC2 instance to use for training.

The following code will launch the training job and store the trained model into S3, **the training process should take ~4 minutes**.

Deposit trained model in SageMaker Model Registry

Once a useful model has been trained and its artifacts properly associated, the next step is to save the model in a registry for future reference and possible deployment. In this section, we will see how you can package your artifacts into a ModelPackage.

▼ Create Model Package Group

A Model Package Group holds multiple versions or iterations of a model. Though it is not required to create them for every model in the registry, they help organize various models which all have the same purpose and provide automatic versioning. A Model Package is a reusable model artifact abstraction that packages all ingredients necessary for inference.

Setup the Model Package Group name according to the `prefix` variable.

▼ Lab 4: Deploy and serve the model

Once training is completed, we can deploy the trained model as a real time endpoint using Amazon SageMaker hosting services. This will allow us to make predictions (or inference) from the model. The steps below will demonstrate how to:

- Evaluate trained model.
- Based on the evaluation results, update the model approval status in the model registry.
- Deploy the model as real time endpoint using SageMaker hosting services.
- Using Dataset and Claims customer dataset prepared as part of Lab 1, run the inference against sample policy id.

This lab will take approximately 10 mins to run.

Evaluate trained model and update status in the model registry

When we registered the model in the model registry, the default status is "Pending Approval" status. In the real-life MLOps lifecycle, a model package gets approved after evaluation by data scientists, subject matter experts and auditors. For the purpose of this lab, we will evaluate the model with test dataset that was created during training process.

Model Evaluation

The following script evaluates the trained model performance metric against a certain threshold value. We will use AUC metric as the model performance metric criteria and set the minimum threshold value as 0.7.

Create SageMaker Processing Job providing:

```
[16]: # Cell 16
# Download evaluation score from the Processing job output
eval_report = "/outputs/evaluation.json"
eval_report_dir = evaluation_job_description['ProcessingOutputConfig']['Outputs'][0]['S3Output']['S3Uri'].replace("s3://", "{bucket}/", '')
s3_client.download_file(bucket, f'{eval_report_dir}/evaluation.json', eval_report)

with open(eval_report) as f:
    model_score_report = json.load(f)

print(model_score_report)

{'binary_classification_metrics': {'accuracy': {'value': 0.973, 'standard_deviation': 'NaN'}, 'auc': {'value': 0.799113090479997, 'standard_deviation': 'NaN'}}
```

▼ Evaluate model AUC score.

Based on the results, update model approval status for that version. Please note, we are using API's to Approve/Reject model approval status. You can also update the approval status with the SageMaker Studio UI.

```
model_package_update_response = sagemaker_botoclient.update_model_package(model_package_update_input_dict)

Model approved!
arn:aws:sagemaker:us-east-1:646975365807:model-package/fraud-detect-demo/1
```

▼ Model deployment

In the previous step, the model status was adjudicated based on AUC of the test dataset. If the AUC was above the defined threshold, then the approval status for that particular version is set to "Approved". At this stage, the model is ready for deployment. Please note, we do not have explicit step to stop the model deployment if the AUC was below threshold. In the real-life MLOPS lifecycle, the model approval status is set to "Rejected" if the model performance is not per the business KPI's and you will either tweak the data processing or training script to improve the model performance.

Once the model is approved, it is ready for deployment. We will deploy the model for real time inference using SageMaker Hosting service. In order to deploy the endpoint, we will

- Define interface endpoint requirements.
- Create Model
- Create Endpoint config
- Create/Update endpoint using newly created EndpointConfig.
- Test the endpoint with sample data.

```
[18]: # Cell 18
# define end point hosting configuration
endpoint_name = f"{prefix}-endpoint"
```

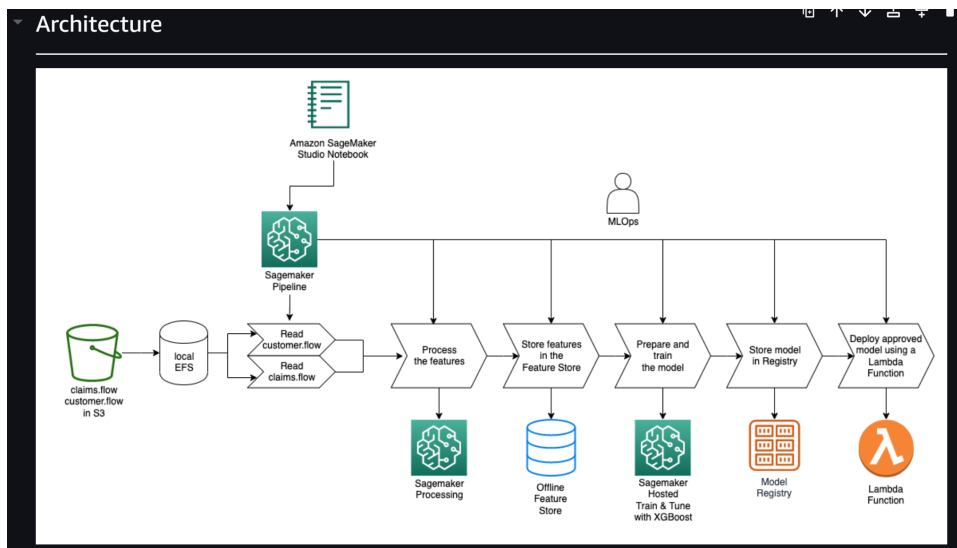
```
[25]: # Cell 24
sample_policy_id = int(policydataset.sample(1)["policy_id"])
customer_claim_df = dataset[dataset["policy_id"] == sample_policy_id].sample(1)
blended_df = customer_claim_df.loc[:, col_order].drop("fraud", axis=1).T.reset_index()
blended_df.columns = ["FeatureName", "ValueAsString"]

data_input = ",".join([str(x) for x in blended_df["ValueAsString"]])
data_input
print(policydataset.sample(1))
```

Unnamed: 0	policy_id	num_insurers_past_5_years	policy_state_ca	\
2009	2009	1519	1	1
incident_day	customer_age	incident_type_collision	\	
2009	2	29	1	
driver_relationship_child	num_witnesses	policy_state_id	...	\
2009	0	0	0	...
incident_month	customer_gender_male	policy_state_az	injury_claim	\
2009	10	1	0	9000.0
collision_type_front	driver_relationship_self	customer_gender_female	\	
2009	1	0	0	
total_claim_amount	collision_type_rear	customer_education		
2009	17500.0	0	4	

[1 rows x 48 columns]

Run prediction on sample claim dataset



Pipelines

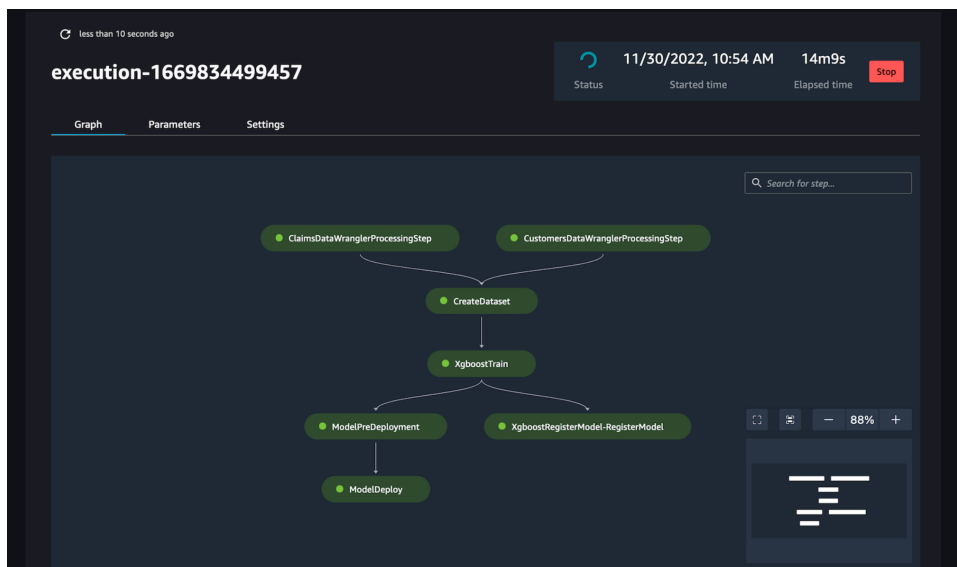
[Show introduction](#)

Search...

Name	Created at	Created by	Tags	Updated at
FraudDetectDemo	3 hours ago	sagemakeruser	sagemaker:user-pr...	3 hours ago

End of results

1 results Results are cached [Refresh](#) Rows: 10 Go to page: 1 Page 1 of 1



LAMBDA FUNCTION

▼ **Function overview** [Info](#) Export to

Diagram **Template**

 **fraud-detection-workshop-DelayLambda-rvBVndxkGpaq**

Related functions:
Select a function ▼

 Layers (0)

+ Add trigger + Add destination

Code source [Info](#)

File Edit Find View Go Tools Window Test ▼ Deploy

Go to Anything (Ctrl-P)

Environment

- fraud-detection-wor
 - cfnresponse.py
 - index.py

index.py

```
1 import json
2 import cfnresponse
3 import time
4 def handler(event, context):
5     time_to_wait = int(event['ResourceProperties']['TimeToWait'])
6     print(f'waiting for {time_to_wait} seconds')
7     time.sleep(time_to_wait)
8     print(f'waiting finished')
9     cfnresponse.send(event, context, cfnresponse.SUCCESS, {}, '')
10
```

